System Design - ConnectSphere Release 1

As-built source: main 0773a721. Code and Jira status are separate from final acceptance.

Lecture basis: Week 4 PDF pp. 58, 60-66 (C4, ER, ADRs); Week 5 PDF pp. 7-15 and 22 (UML). DFDs and user flows are supplementary.

Known limits: single-role runtime, Coordinator-triggered initial safety submission, eligible automatic re-entry and no frequent worker cron declared. See docs/design/README.md for scope and discrepancies.

01. C4 system context

02. C4 containers and deployment

03. C4 API components

04. Release 1 user journey

05. Event lifecycle state machine

06. Data flow context (Level 0)

07. Data flow decomposition (Level 1)

08. ERD: identity, Events and Sessions

09. ERD: venue, equipment and technical support

10. ERD: registration, safety and durable notifications

11. UML code view: lifecycle repository boundary

12. Sequence: atomic Draft submission and refusal

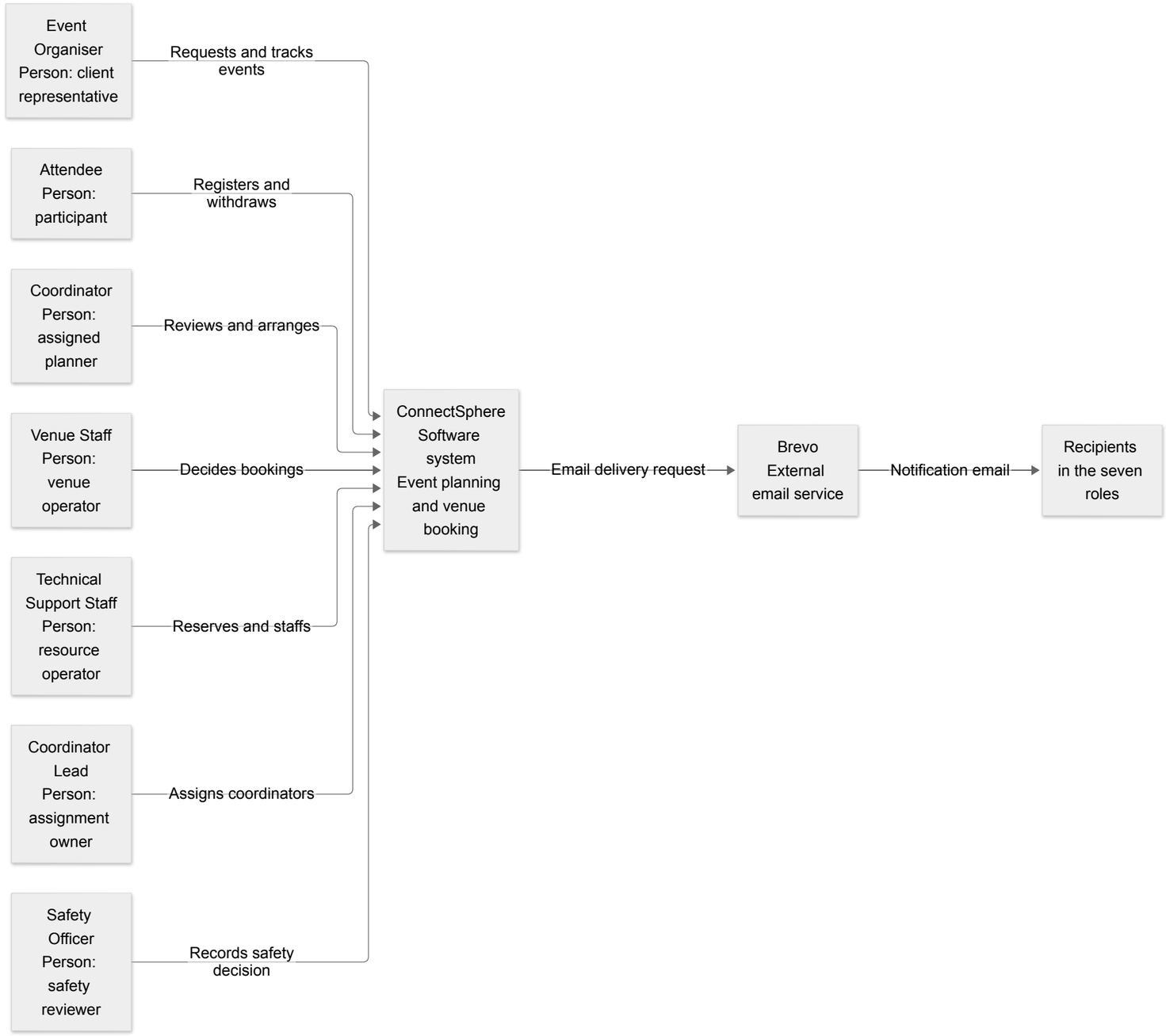
13. Sequence: transactional outbox to email

C4 system context

Seven supported roles use one system. Brevo delivers email. Supabase PostgreSQL and Upstash are managed infrastructure, expanded in C2; no SMS, payments or third-party calendar integration is claimed.

Notation: C4 C1; typed actors, system boundary and labelled relationships.

Week 4 PDF pp. 58, 60 (printed slides 61 and unnumbered C1); api/; frontend/src/app/routes.tsx. Lecture references use PDF page numbers, not necessarily printed slide numbers.

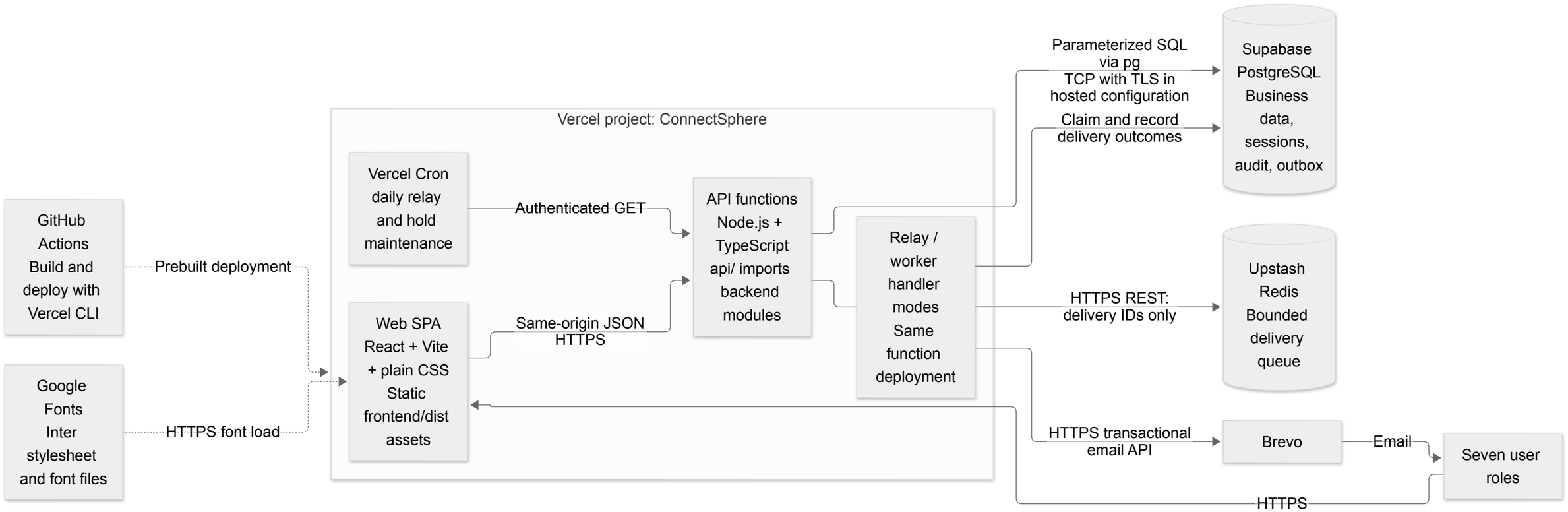


C4 containers and deployment

The browser SPA and serverless API deploy through one Vercel project. Relay and worker are logical handler modes in the same API deployment, not two always-running daemons. Daily Vercel schedules cover relay and hold/completion maintenance; a frequent worker scheduler is not declared in vercel.json.

Notation: C4 C2; container means runtime application or data store, not Docker.

Week 4 PDF p. 61; vercel.json; frontend/package.json; backend/src/providers/; api/cron/outbox-relay.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.

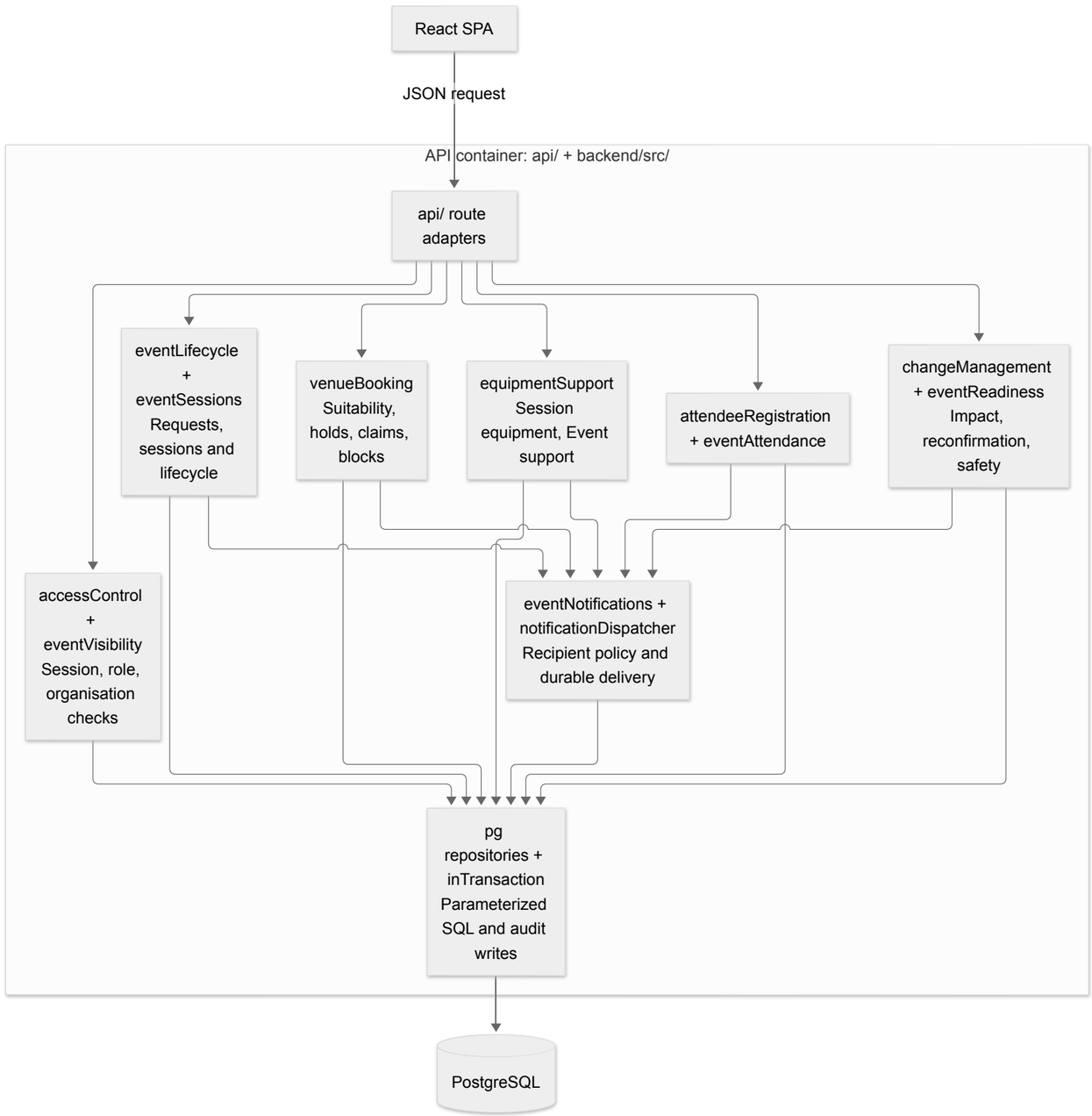


C4 API components

These are code boundaries, not microservices. Handlers authenticate and route to domain modules. Modules use raw SQL, shared transactions, audit helpers and transactional notification hooks; there is no ORM. Session authoring, readiness and attendance are separate implemented modules.

Notation: C4 C3; selected modules within API container.

Week 4 PDF p. 62; api/events.ts; api/venues/index.ts; backend/src/modules/. Lecture references use PDF page numbers, not necessarily printed slide numbers.

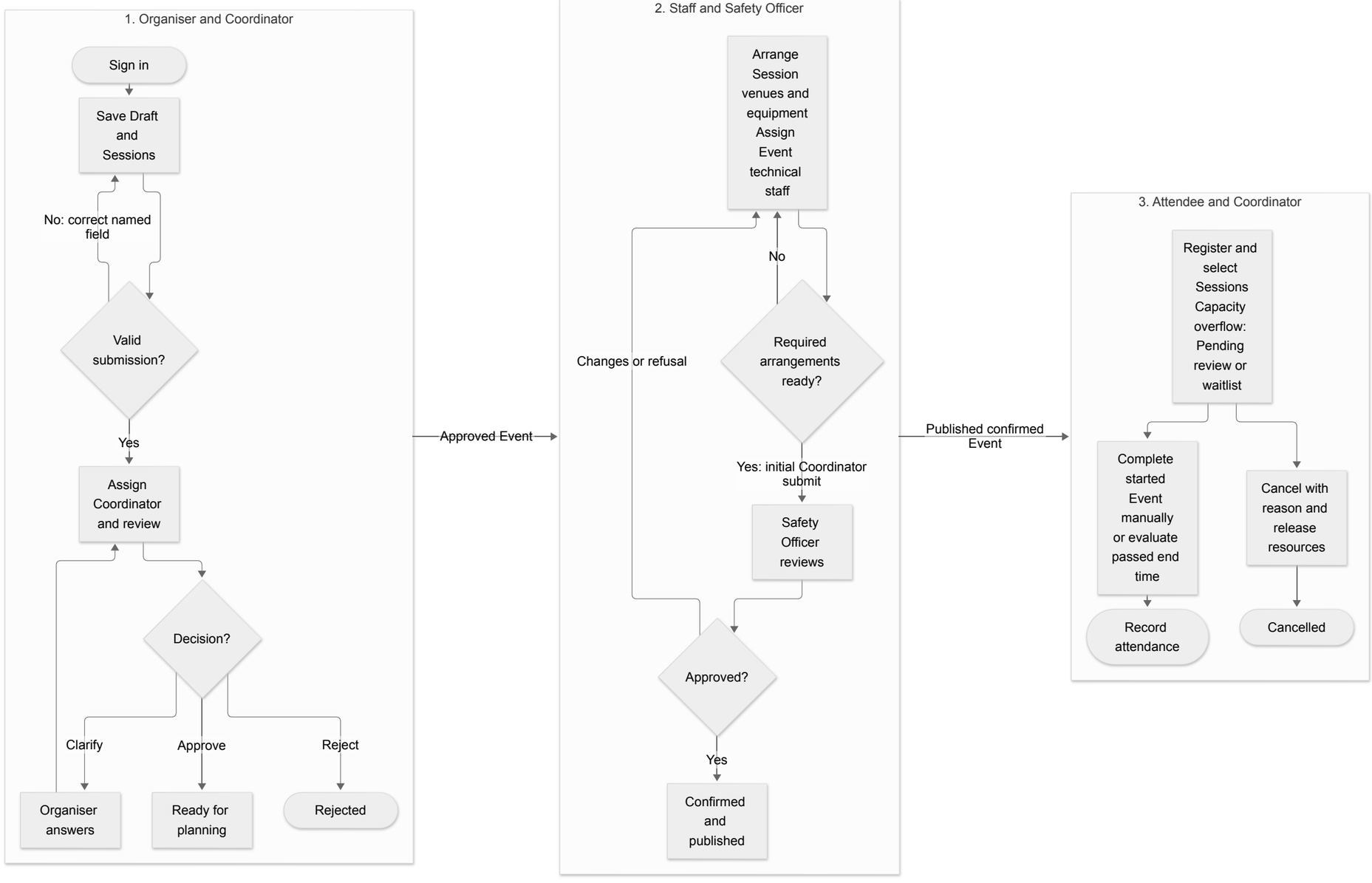


Release 1 user journey

This is a navigation/business journey, not a DFD. As built, initial Safety Review is submitted by the Coordinator once ready; re-entry after reconfirmation is automatic. This differs from the broader T-82 automatic-entry target and is explicitly recorded in the discrepancy inventory. The Safety Officer approves confirmation. Registration is Event-level with selected Sessions; the coordinator records attendance after completion. Pending Jira gates are listed in the scope inventory.

Notation: Flowchart: rounded start/end, decision diamonds, labelled paths.

Supplementary user-flow view; Week 5 PDF p. 7 motivates focused behaviour views; frontend/src/app/routes.tsx; backend/src/modules/eventReadiness/. Lecture references use PDF page numbers, not necessarily printed slide numbers.

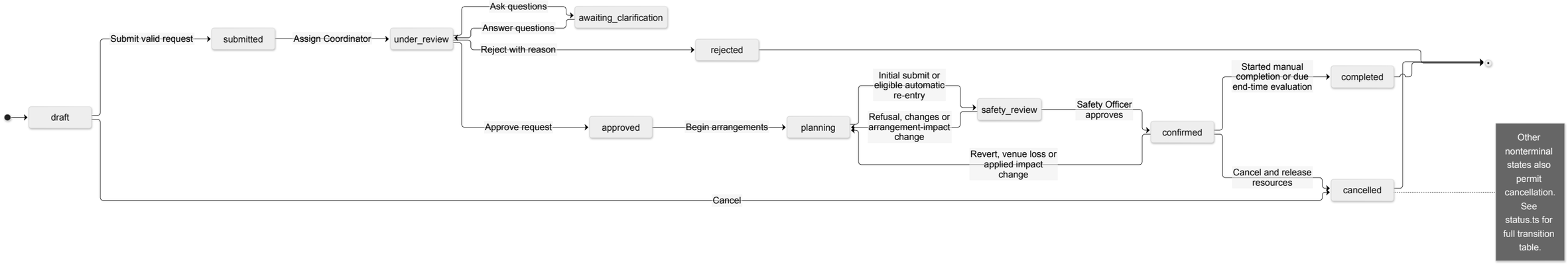


Event lifecycle state machine

The diagram follows the implemented status vocabulary. Generic transition validation is supplemented by role and arrangement checks. Rejected, Cancelled and Completed are terminal. Cancellation routes exist from nonterminal states; they are grouped in a note to avoid duplicate crossing lines.

Notation: UML-style state diagram; transition labels describe triggers.

Supplementary lifecycle view; backend/src/modules/eventLifecycle/status.ts; eventReadiness/autoSafetyReview.ts; completion.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.

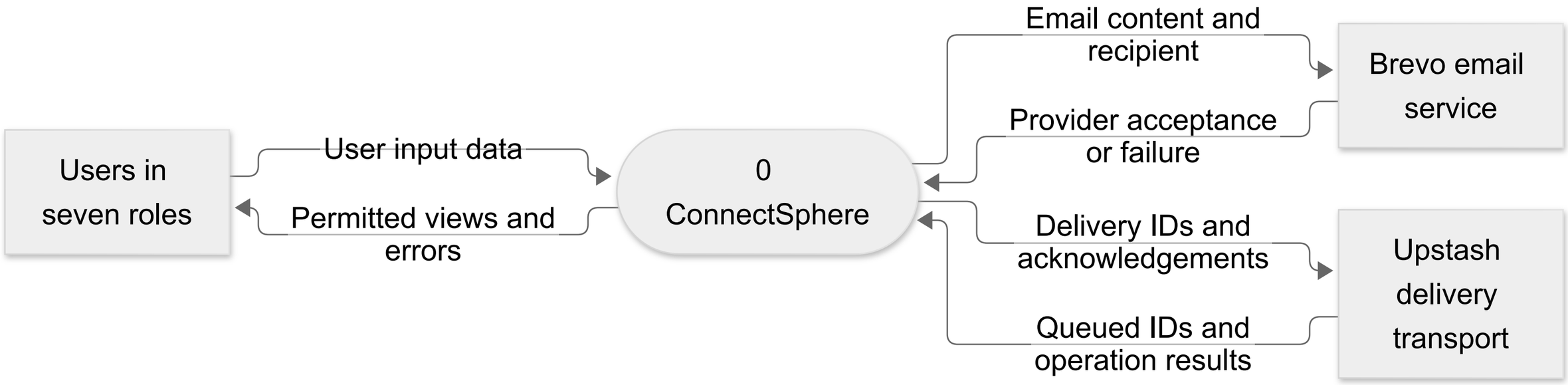


Data flow context (Level 0)

One process represents the application and its server-side persistence boundary. This is a logical data-flow view, not deployment topology. Every external flow is balanced in Level 1; scheduler triggers are intentionally omitted because they are control flow, not business data.

Notation: Supplementary DFD: rectangles external entities; rounded process; noun-labelled data arrows.

No DFD levels or notation were taught in the supplied lecture PDFs. Added at user request; api/; backend/src/providers/. Lecture references use PDF page numbers, not necessarily printed slide numbers.

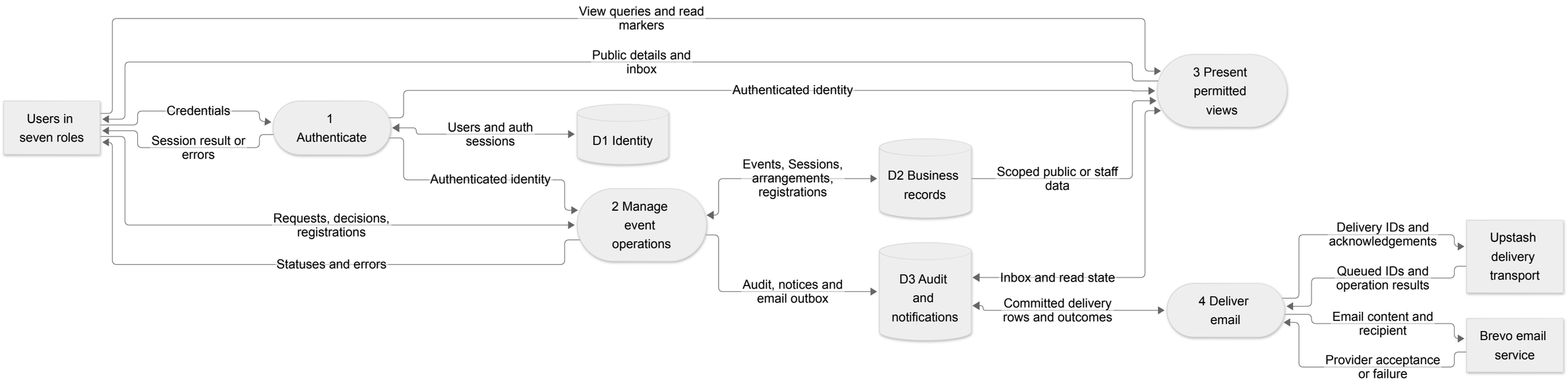


Data flow decomposition (Level 1)

P1 checks credentials and sessions, P2 applies domain rules, P3 reads public/in-app results, and P4 handles email delivery. D1-D3 are logical groups in the same PostgreSQL database, not three databases. There are no direct external-entity-to-store arrows.

Notation: Supplementary DFD: numbered processes; cylinder data stores; labelled data flows.

User-requested supplementary notation; backend/src/modules/accessControl/; eventLifecycle/; notificationDispatcher/. Lecture references use PDF page numbers, not necessarily printed slide numbers.

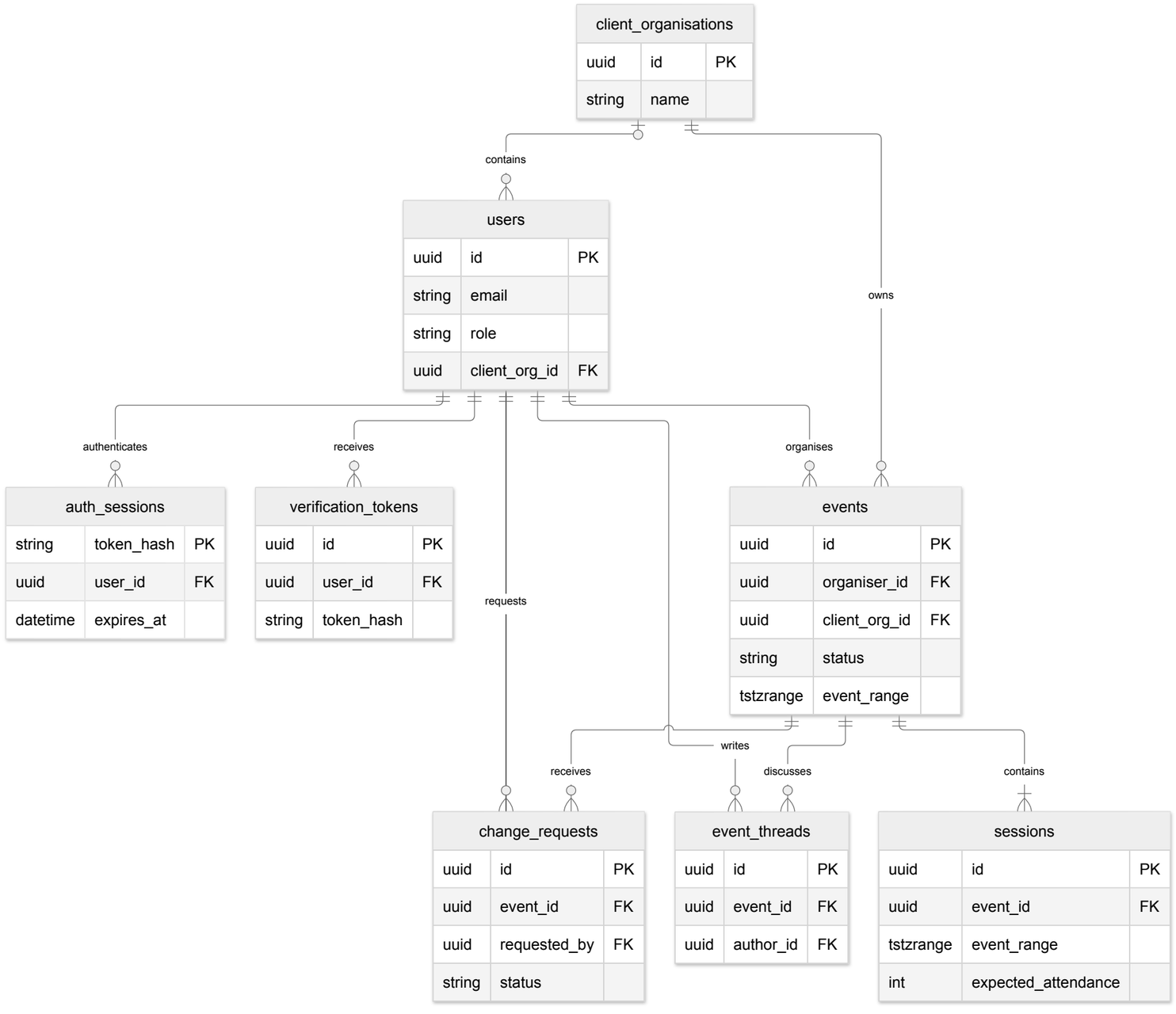


ERD: identity, Events and Sessions

This selected physical view deliberately shows users.role as built, not the unimplemented USER_ROLES target in ADR-019. An Event has one or more Sessions by application invariant. The physical FK alone permits zero children temporarily; join-table details are in the full schema inventory.

Notation: Mermaid crow-foot ER; PK/FK and optionality; selected physical tables.

Week 4 PDF p. 63 mentions ER at code level; migrations 0001, 0003, 0006, 0014, 0024. Lecture references use PDF page numbers, not necessarily printed slide numbers.

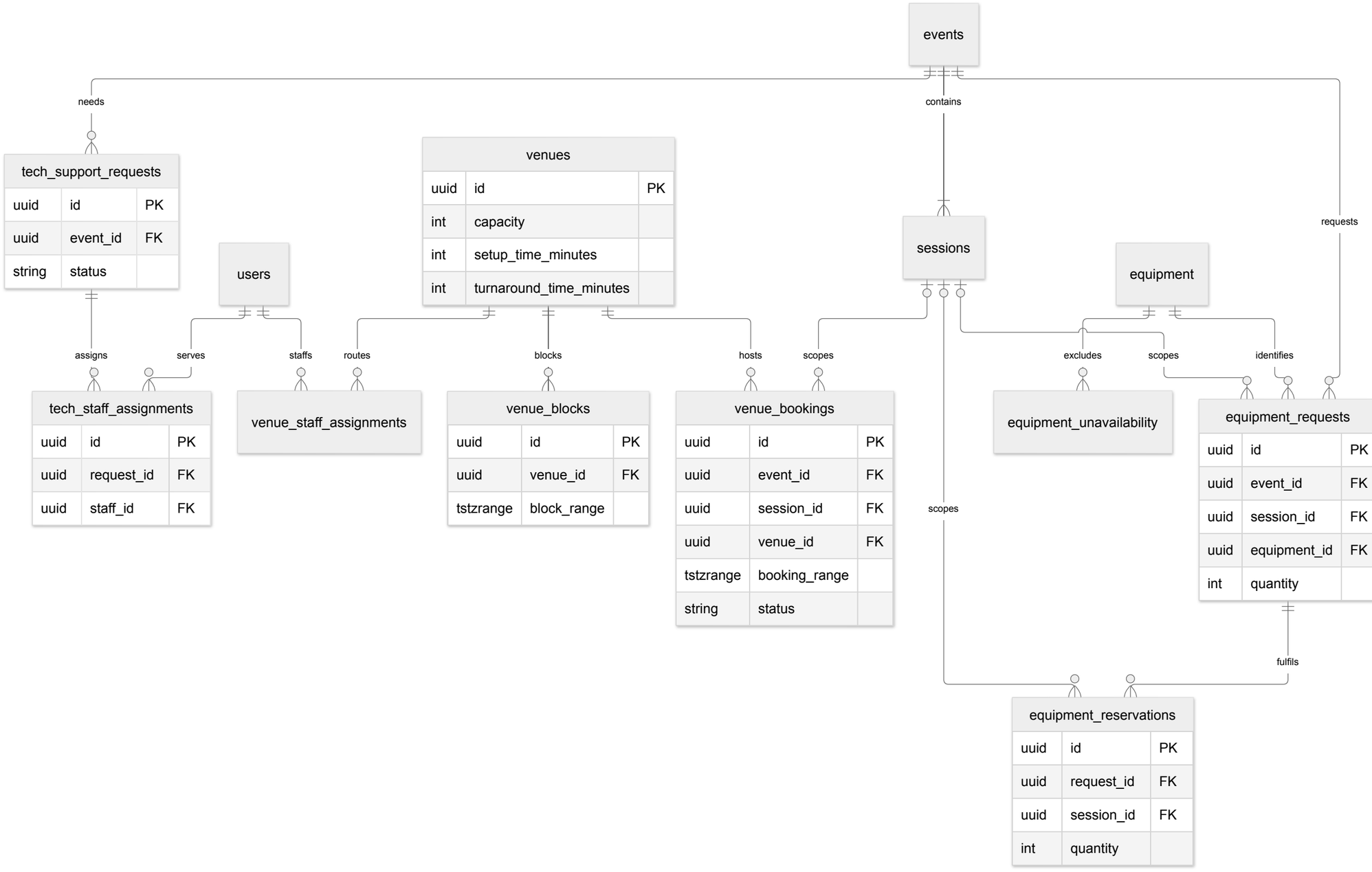


ERD: venue, equipment and technical support

Venue bookings are linked to Sessions. Equipment requests/reservations carry Session links on new multi-Session paths; legacy Event-level rows remain possible. Technical-support assignment remains Event-level. Venue staff routing is setup data; no staff-roster UI is claimed.

Notation: Mermaid crow-foot ER; selected physical tables and nullable Session links.

Week 4 PDF p. 63; migrations 0001, 0014, 0018, 0019, 0025, 0026; equipmentSupport/sessions.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.

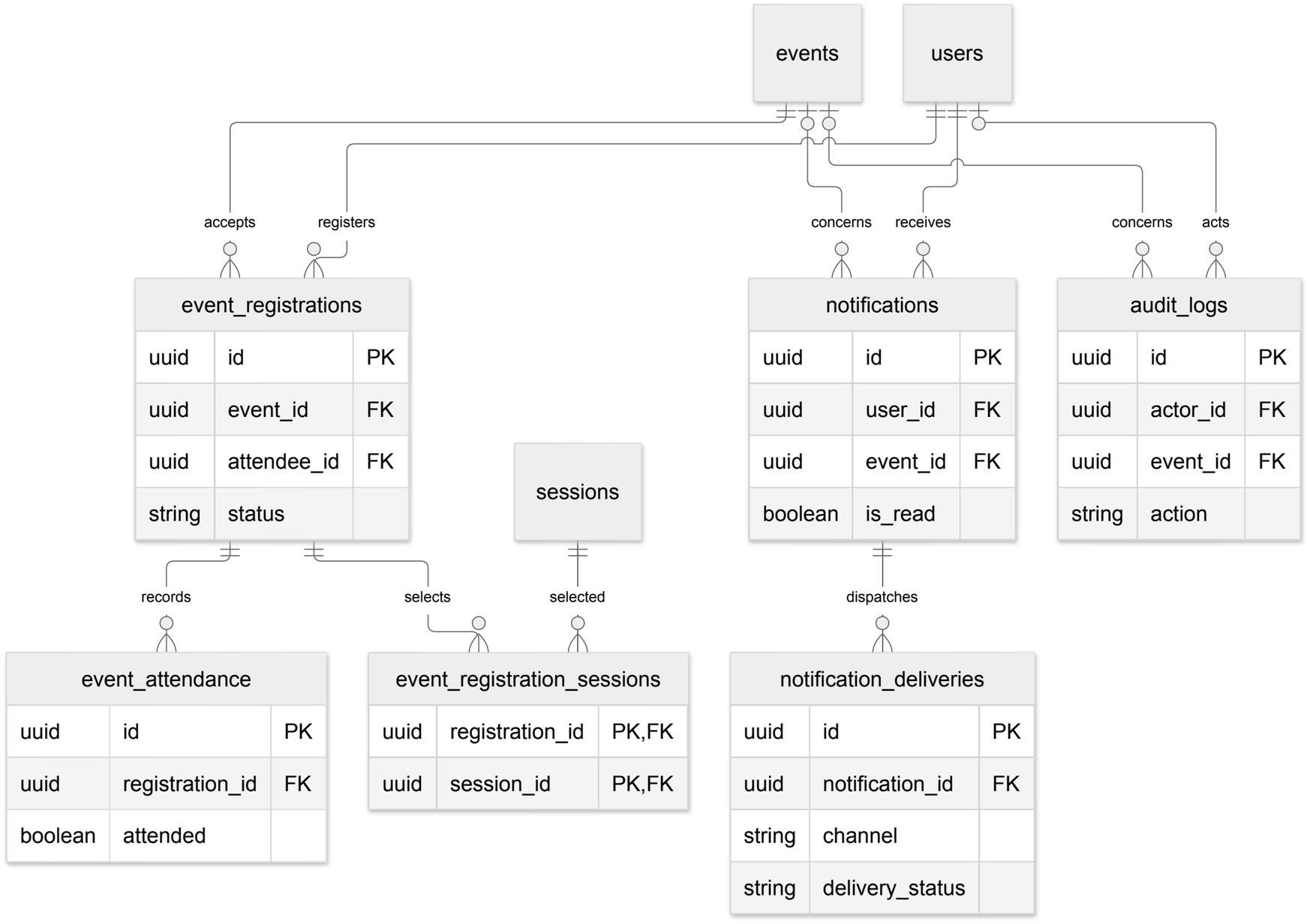


ERD: registration, safety and durable notifications

An Event registration selects Sessions through a join table. Attendance belongs to a registration. Notifications are user-facing rows; channel deliveries carry durable transport state. Audit event and actor links are nullable so history survives relevant deletion. Keyless entities in the drawing are relationship anchors, not missing physical definitions. Safety decisions and factors are recorded in audit_logs (field_changed=safety_check); there is no physical safety_checks table.

Notation: Mermaid crow-foot ER; selected physical tables.

Week 4 PDF p. 63; migrations 0001, 0002_durable_notification_dispatch, 0017, 0023, 0024. Lecture references use PDF page numbers, not necessarily printed slide numbers.

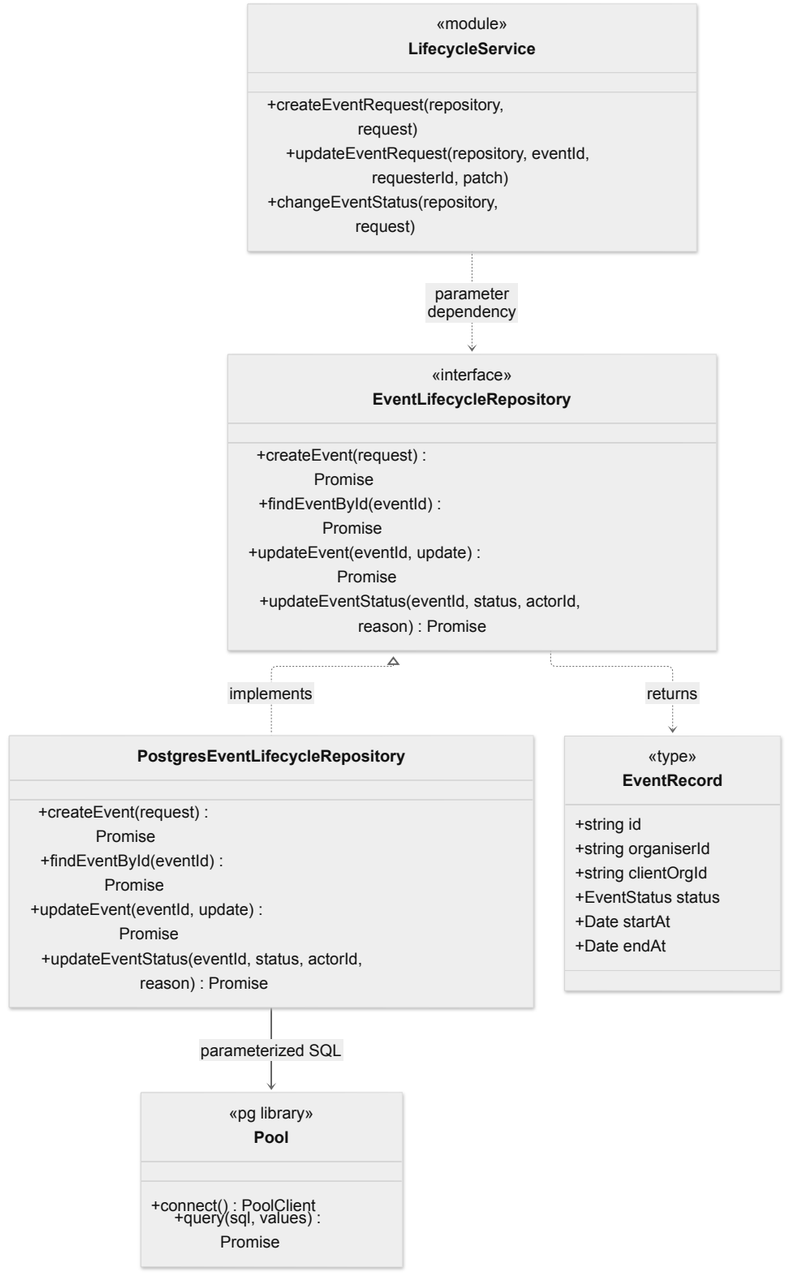


UML code view: lifecycle repository boundary

The implementation uses exported functions and repository classes, not invented service classes. The service module is represented with a module stereotype, EventLifecycleRepository as an interface, and EventRecord as a data type. Dependencies are deliberately focused on the draft submission slice.

Notation: UML class/interface realization and dependency; TypeScript structural types.

Week 5 PDF pp. 7-13; backend/src/modules/eventLifecycle/service.ts, repository.ts, types.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.

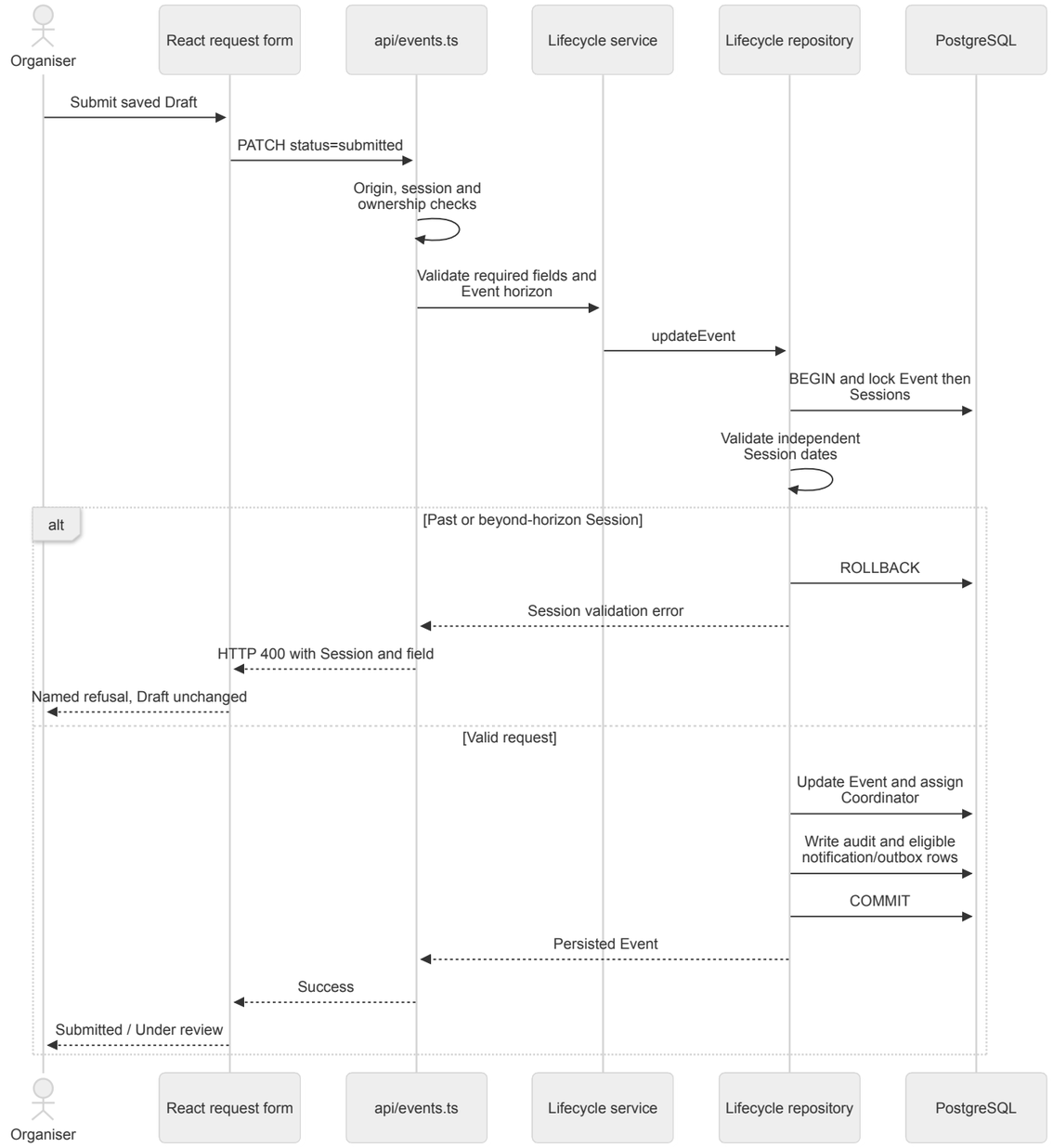


Sequence: atomic Draft submission and refusal

Event and Session locks prevent a queued Session writer from bypassing submission. The same Event lock is acquired by draft Session writes. A failed date check rolls back and returns a structured error; successful business, audit and notification changes commit together. No provider call occurs inside the transaction.

Notation: UML sequence; lifelines, messages, alt branch, transaction ordering.

Week 5 PDF pp. 15, 22; api/events.ts; eventLifecycle/repository.ts; eventSessions/repository.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.



Sequence: transactional outbox to email

PostgreSQL is authoritative; Redis carries only delivery IDs. Relay and worker are separate invocations of handler modes, with flags and internal-secret checks. Send leases deduplicate work; ambiguous outcomes are retained as uncertain rather than blindly resent. This diagram specifies built code, not a claim of continuous production scheduling.

Notation: UML sequence; asynchronous phases and provider outcome branch.

Week 5 PDF p. 15; notificationDispatcher/dispatch.ts; postgres.ts; providers/durableRedis.ts; api/cron/outbox-relay.ts. Lecture references use PDF page numbers, not necessarily printed slide numbers.

